LECTURE 22

Horizontal Scaling & Distributed Backend Architecture

Detailed backend engineering notes • GitHub-ready Markdown companion

Central principle
Make application instances interchangeable, distribute work intelligently, and introduce distributed infrastructure only when its benefits justify its complexity.

Continuation of Lecture 21. This lecture expands horizontal scaling into stateless application design, load balancing, database scaling, CDNs/edge computing, asynchronous work, monoliths vs microservices, autoscaling, and serverless computing.

1. Statelessness: the key to horizontal scaling

Horizontal scaling (scale out) means adding more instances of an application rather than only increasing the capacity of one machine. Multiple instances run the same application code, and incoming requests are distributed among them.

Horizontal scaling works reliably when any instance can handle a request without depending on private state stored only inside another instance.

- **Stateful instance:** holds information that is exclusive to that server, such as an in-memory session or a file stored only on its local disk.
- **Stateless application tier:** does not rely on instance-local state to process a request. Shared/persistent state is stored in systems accessible to all instances.

A useful test: if instance B disappears, can instances A and C continue to serve users correctly? If B held the only copy of a session or uploaded file, the architecture is not safely stateless.

Externalize shared state

State/data • Fragile instance-local approach • Shared approach discussed in the lecture

Sessions • Store sessions in instance A's RAM • Store sessions in shared Redis/in-memory data store

Uploaded files • Save file to instance A's local SSD • Store in shared object/blob storage such as S3 or Cloudflare R2

Application data • Keep a database file local to one application instance • Use a centralized/shared database such as PostgreSQL or a managed database

Other shared information • Keep a private copy on one instance • Put it in a shared service reachable by every instance

Example: session failure

- A user authenticates through instance A.
- A creates a session and stores it only in A's memory; the browser receives a session cookie.
- The next request is routed to instance B.
- B cannot find the session in its own memory and may return `401 Unauthorized`.
- The user appears to have been logged out even though the session was created successfully.

Fix: Store the session in a shared store such as Redis. Every application instance can then look up the same session identifier.

The same principle applies to uploaded files: if one instance stores a file locally, a later request routed to another instance may not find it. Use shared object storage.

Thumb rule: With horizontal scaling, do not make correctness depend on information stored only inside one application instance.

2. Load balancers

A **load balancer (LB)** is the middle layer that receives client requests, chooses an application instance, forwards the request, receives the response, and returns it to the client.

Typical flow:

`Client → Load Balancer → Application Instance → Load Balancer → Client`

The important decision is which instance should receive the next request. The load-balancing algorithm determines this.

2.1 Round robin

Requests are distributed in a repeating order:

`A → B → C → A → B → C → ...`

Works best when:

- Instances have similar capacity.
- Requests have roughly similar processing cost.
- Traffic is reasonably balanced.

Limitation: Round robin does not inherently understand request cost. If some requests take 200 ms and others take 2 seconds, a server may receive several expensive requests and become overloaded even though requests are distributed evenly by count.

2.2 Weighted round robin

Instances receive traffic in proportion to configured weights/capacity.

Example:

- Instance A: 8 GB RAM and 4 CPU cores.
- Instances B and C: 4 GB RAM and 2 CPU cores each.

A can be assigned roughly twice the weight of B or C, so it receives more requests. This accounts for different instance capacities, but still does not necessarily understand the cost of each individual request.

2.3 Least connections

The load balancer checks the number of active connections and prefers the instance with the fewest.

Why this can help:

- A lightweight request may finish quickly, freeing its connection.
- An expensive request may remain active while it calls an external service or performs a database write.
- The instance handling the long-running request continues to show an active connection, so subsequent requests can be sent to less-busy instances.

A weighted least-connections variation can account for differences in server capacity.

The lecture uses HTTP request/response connections as an intuitive model. Actual connection behavior depends on HTTP version, keep-alive, multiplexing, and the load balancer's implementation; connection count is a load signal, not a perfect measurement of CPU or total work.

2.4 Least response time

Prefer instances that are returning responses faster. This can steer traffic away from servers that are struggling, although response time can be influenced by request mix and transient conditions.

2.5 Resource-based routing

Use signals such as CPU or memory utilization to prefer instances with more available resources.

Algorithm comparison

Algorithm • Decision basis • Main strength • Main limitation

Round robin • Rotating order • Simple and predictable • Ignores request cost and instance load

Weighted round robin • Configured capacity weights • Supports unequal instance sizes • Still mostly distributes by count/weight

Least connections • Active connection count • Adapts to long-lived/in-flight work • Connections are only an approximation of load

Least response time • Observed response speed • Can avoid slower instances • Past response time may not predict the next request

Resource-based • CPU, memory, other resource signals • Uses current capacity signals • Requires reliable telemetry and can react to noisy signals

3. Health checks and unhealthy instances

A load balancer must avoid sending user traffic to a failed instance.

Health checks are periodic test requests sent by the load balancer to application instances. A simple health endpoint may return `200 OK` when the instance is healthy.

Conceptual flow:

- The LB periodically probes each instance.
- A healthy instance returns the expected success response.
- If an instance fails the configured health-check policy, the LB marks it unhealthy and stops routing normal user traffic to it.
- The LB continues checking it.
- When the instance passes the recovery policy, it can be returned to the healthy pool.

Without health checks, round robin may keep sending requests to a dead instance, causing users to receive errors such as `502` or `503`.

Important implementation nuance: Production health checks normally use thresholds, timeouts, consecutive failures/successes, and carefully designed readiness/liveness endpoints. A single failed probe should not always cause immediate removal; transient network glitches can occur. A readiness check should indicate whether the instance can safely receive traffic, not merely whether its process exists.

4. Scaling databases: why it is harder

Application instances can often be duplicated after state is externalized. A database is different: it owns persistent data, and multiple database instances must coordinate so that reads and writes remain correct.

The lecture introduces two common strategies:

- **Read replicas / replication**
- **Sharding / partitioning**

5. Read replicas

A **primary database** accepts writes. One or more **read replicas** maintain copies of the primary's data and serve read queries.

Conceptual routing:

- `INSERT`, `UPDATE`, `DELETE` → primary.
- `SELECT` → read replica, where appropriate.

Benefits

- Distributes read workload away from the primary.
- Can reduce primary resource utilization.
- Replicas can be placed nearer to users in different geographic regions, reducing read latency when the application tier is also deployed appropriately.

The lecture illustrates a read-heavy application where perhaps 70% of requests are reads and 30% are writes. These percentages are examples, not universal workload ratios.

Replication lag and stale reads

Replicas are not necessarily updated at the exact same instant as the primary. The delay is called **replication lag**.

Example:

- A user changes their profile name from A to B.
- The write succeeds on the primary.
- The application immediately performs a read, routed to a replica.
- Replication has not caught up yet.
- The replica returns the old value A.

The user sees a successful save followed by stale data. In sensitive workflows, stale reads can cause more serious correctness problems.

Strategies mentioned for read-after-write consistency

- **Route related reads to the primary:** after a write to a particular entity/table, send subsequent reads for that data to the primary for a period or within the relevant workflow.
- **Wait for replication:** monitor replication lag and delay a read until the replica has caught up, where the database/provider exposes a reliable way to do so.
- **Delay the follow-up read:** the client can wait briefly before fetching again. This is a UX workaround, not a universal consistency guarantee.

Choose based on the consistency requirements and acceptable latency. Do not assume every read replica provides immediate consistency.

6. Sharding (partitioning across database instances)

Sharding means dividing a large dataset across multiple physical database instances. Each shard stores only a portion of the total data.

Example: an e-commerce `orders` table becomes extremely large. It can be split into separate database instances based on a chosen key, such as an order date range.

- Shard 1: orders from January through June.
- Shard 2: orders from July through December.

A routing layer determines which shard contains the requested data and sends the query to that shard.

Sharding key

The **sharding key** is the attribute used to decide where a row belongs. Choosing it is a major design decision.

A good sharding key should align with common access patterns and distribute data/workload suitably. The lecture's date-range example is illustrative; real systems must also consider skew, growth, cross-shard queries, rebalancing, and operational complexity.

Why shard?

- Each database instance holds less data than one enormous database.
- Work and request capacity can be spread across multiple database instances.
- Queries targeted to a shard may operate over a smaller dataset.

Sharding vs read replicas

Read replicas • Sharding

Copies the same dataset to multiple instances • Splits the dataset across instances

Replicas commonly serve reads; primary handles writes • Each shard owns a subset of the data

Main goal: scale reads and improve geographic read latency • Main goal: distribute data and database workload

Replication lag can produce stale reads • Routing and cross-shard operations add complexity

Managed/distributed databases

The lecture mentions managed/distributed database products such as PlanetScale, Neon, CockroachDB, and YugabyteDB as examples of systems/providers that address parts of replication, sharding, and distributed database operation.

The practical recommendation is not to build complex database infrastructure from scratch without the expertise to operate it. Understand the concepts so you can make informed choices and configure managed services, backups, replicas, regions, and scaling options.

7. Content Delivery Networks (CDNs)

A **CDN (Content Delivery Network)** caches content at geographically distributed edge locations, also called **Points of Presence (PoPs)**.

Why CDNs matter: physical distance

Network latency is constrained by the speed at which signals travel through physical networks, including fibre-optic and undersea cables. A user far from the origin server experiences network round-trip latency even if the server code is highly optimized.

The lecture gives a Tokyo-to-US example: a request may incur around 100 ms of round-trip network latency before application processing is considered. This is an illustrative estimate; real latency depends on routing, network conditions, and endpoints.

How a CDN helps

Instead of every request travelling to the origin server:

- Content is cached at a nearby edge location.
- The user requests that content.
- A nearby CDN PoP serves a cache hit.
- Only misses or content requiring origin processing need to go back to the origin.

Benefits:

- Lower latency for cached content.
- Less traffic and work reaching the origin server.
- Distributed delivery capacity.

Common CDN content

- Static JavaScript and CSS bundles.
- HTML/static site assets.
- Images, videos, and fonts.
- Selected API responses when caching is safe and appropriate.

Cache invalidation / purging

When content changes, a CDN may need to purge cached copies so users do not keep receiving stale content. The lecture describes tag-based purging: tag related content (for example, a user's blog posts) and invalidate entries associated with that tag after an update.

Caching API responses requires particular care around authorization, user-specific content, cache keys, freshness, and privacy.

8. DDoS protection and CDN security

A **denial-of-service (DoS)** attack attempts to overwhelm a service with traffic. A **distributed denial-of-service (DDoS)** attack uses many sources, often compromised devices/bots, to generate traffic.

Potential impact:

- The origin becomes overloaded or unavailable.
- Autoscaling may create more instances and unexpectedly increase infrastructure cost.
- Users experience errors or severe latency.

A CDN/security provider placed in front of the origin can:

- Absorb and distribute traffic across its network.
- Detect suspicious traffic patterns.
- Apply filtering/challenges/rate controls.
- Serve cache hits without sending every request to the origin.

A CDN is not a guarantee against every attack; protection depends on configuration, provider capacity, and the attack type.

9. Edge computing

Edge nodes are CDN locations close to users. Traditional CDN use is primarily serving cached content. **Edge computing** means running some processing at or near those edge locations before forwarding or returning a response.

Examples from the lecture

- **Authentication/rejection:** validate a suitable session/token at the edge and reject clearly unauthorized requests close to the user, reducing unnecessary traffic to the origin.
- **User customization:** use location or request headers such as preferred language to select localized content.
- **Routing/validation:** perform lightweight request decisions before traffic reaches the primary application.

Why not run everything at the edge?

Edge environments have constraints:

- Less CPU and memory than full data-centre servers.
- Restricted filesystem access and runtime capabilities.
- Some protocols and long-lived connection patterns may not be supported.
- The environment is optimized for low-latency, lightweight work.

The lecture names Cloudflare Workers and V8 isolates as examples of edge execution technology. The practical model is to move suitable lightweight logic to the edge while retaining robust application processing in origin data centres.

10. Asynchronous processing and background jobs

Asynchronous processing reduces **perceived request latency** by moving work that does not need to finish before the user receives a response into a background queue.

Synchronous vs asynchronous example: inviting a team member

Synchronous flow:

- Validate the invite.
- Check membership in the database.
- Store a pending invitation.
- Call an external email provider.
- Wait for the email provider's response.

- Return success to the user.

The external email call can add hundreds of milliseconds or more.

Asynchronous flow:

- Validate the request and save the pending invitation.
- Enqueue an email-sending job.
- Return a response to the user.
- A worker later consumes the job and calls the email provider.

The user sees a faster response because the request no longer waits for email delivery.

Queue architecture

`API / Producer → Queue → Worker / Consumer → External service or task`

- **Producer:** creates and enqueues the job.
- **Queue:** stores pending work.
- **Worker/consumer:** retrieves and executes jobs.
- Workers can run in the same application deployment or as separately scalable processes.

The lecture mentions Redis-based queues, RabbitMQ, Kafka/event processing, and BullMQ in the Node.js ecosystem.

Suitable background tasks

- Sending emails and notifications.
- Video encoding and thumbnail generation.
- Image resizing/processing.
- Large account/data deletion workflows.
- Other work where the user does not need the full result immediately.

Example: account deletion

Deleting a large account may require removing data from many tables and take several seconds. Instead of making the user wait for every deletion query:

- Authenticate/validate the request.
- Enqueue a deletion job.
- Respond that the deletion request has been accepted/started.
- A worker performs the cleanup in the background.

Important API semantics: If the operation is not yet complete, avoid falsely claiming that deletion is already finished. A response such as `202 Accepted` with a status/resource for tracking is often more accurate. The lecture's simplified example describes returning a success response and logging the user out; production systems should communicate the true operation state.

Engineering safeguards

Background jobs may be retried or delivered more than once. Design handlers to be idempotent where possible, record job status, handle permanent vs transient failures, apply retry/backoff policies,

and monitor queue depth and job age. These are practical implementation considerations that support the lecture's queue-based design.

11. Monoliths vs microservices

Monolith

A **monolith** is an application delivered as one deployable unit. Authentication, payments, orders, notifications, webhooks, and other modules may be separated in the codebase, but they are built and deployed together.

Benefits:

- One codebase/deployment unit.
- Straightforward development and refactoring.
- Easier local testing and debugging in many cases.
- Internal module calls can be ordinary in-process calls rather than network calls.

A monolith can still be horizontally scaled by running multiple copies of the application.

Why consider microservices?

The lecture emphasizes that microservices are often about **scaling teams and organizational boundaries**, not automatically improving raw machine performance.

Potential motivations:

- **Independent deployment:** teams can release one service without coordinating a release of every module in a single deployable unit.
- **Independent scaling:** resource-heavy payment/order processing can scale separately from a lightweight notification service.
- **Different technology stacks:** different services can use technology suited to their workloads and libraries.

The lecture suggests microservices become more relevant in very large organizations/teams (it gives 100–200+ developers as an illustrative scale), but team size alone is not a universal threshold.

Microservice costs

- **Network latency:** in-process function calls become HTTP/gRPC network calls.
- **Network failures:** timeouts, retries, partial failures, and service availability must be handled.
- **Debugging complexity:** one request may pass through several services; logs from each service and distributed tracing are needed.
- **Data consistency:** services may own separate databases; updates across services introduce consistency and replication challenges.
- **Operational complexity:** more services mean more deployments, monitoring, configuration, and failure modes.

When might microservices make sense?

- Teams are large enough to need clear ownership boundaries.
- Components have genuinely different scaling requirements.

- Different components have strong technology/runtime requirements.
- Independent deployment is valuable enough to justify the network and operational complexity.

Default principle: Start with a well-structured monolith unless there is a clear reason to split it. Microservices are not a mandatory step for every growing application.

12. Traditional servers and capacity planning

In a traditional/serverful model, developers provision a VM or machine, select CPU/RAM/storage/network capacity, install/configure the operating system and software stack, deploy the application, and maintain the server.

Capacity planning problem

Capacity must be estimated before traffic arrives:

- **Under-provisioning:** a traffic spike overwhelms the available CPU/memory and causes slow responses or failures.
- **Over-provisioning:** the application pays for capacity that remains unused.

The lecture's dollar figures are illustrative and should not be treated as pricing benchmarks.

13. Autoscaling

Autoscaling adjusts the number or size of application instances based on configured signals such as CPU or memory utilization.

Example:

- Start with two instances.
- Traffic rises and resource usage crosses a configured threshold (the lecture uses 70% as an example).
- The platform starts another instance.
- The load balancer distributes traffic across the larger pool.

Limitations

- Autoscaling is often reactive: it detects load after it begins.
- Starting new instances takes time.
- A sudden spike may overload the system before new capacity becomes ready.
- A minimum number/size of instances still incurs baseline cost.

Autoscaling is useful, but it does not replace headroom, load testing, monitoring, or capacity planning.

14. Serverless computing

Serverless does not mean there are no servers. Servers, machines, and runtimes still execute the code; the cloud provider manages much of the infrastructure lifecycle so developers focus on functions and events.

Traditional serverful vs serverless

Traditional serverful • Serverless

Developer manages/provisions server capacity and runtime environment	Provider manages execution infrastructure
Server is generally kept running and ready	Function execution is triggered by requests/events
Pay for provisioned/running capacity over time	Pricing is commonly tied to invocations and execution/resource usage
Capacity planning and server management are explicit responsibilities	Provider abstracts much of this, subject to platform limits

API gateway and function execution

A common HTTP flow:

- Client sends an HTTP request to an API gateway.
- The gateway maps the route/event to a function.
- The provider runs the function in its managed execution environment.
- The function returns a result.
- The execution environment may be reused for later requests or eventually reclaimed.

Functions can also be triggered by non-HTTP events, such as a new queue message, file upload, or database change.

Cold starts

A **cold start** is the startup delay when the provider needs to initialize an execution environment before running a function.

Causes discussed:

- Provisioning/starting the execution environment.
- Runtime initialization (language/runtime and dependencies).

Possible mitigations/technology approaches mentioned:

- Keep a small number of instances warm (but this can reduce cost savings).
- Use faster lightweight execution environments.
- The lecture mentions Firecracker microVM technology and Cloudflare Workers' V8 isolates as examples.

Cold-start behavior varies by provider, runtime, memory configuration, dependencies, and workload. The transcript's timing figures are illustrative/provider-specific, not guaranteed.

Serverless limitations discussed

- **Execution duration limits:** functions may have maximum runtime limits; long-running work may not fit.
- **Cold-start latency:** first invocation can be slower.
- **Stateless execution model:** local process memory and machine identity should not be treated as durable shared state.
- **Connection behavior:** long-lived TCP/database connections and WebSocket-style interactions need careful design.
- **Platform constraints:** runtime, filesystem, networking, and resource limits vary by provider.

Good use cases

- Image resizing and video processing triggered by uploads.
- Event-driven tasks.
- Queue consumers and background processing.
- Infrequent or bursty workloads where keeping dedicated servers running continuously is wasteful.

Less suitable cases highlighted by the lecture

- Latency-sensitive user-facing flows where cold-start variability is unacceptable.
- Long-running procedures beyond function limits.
- Applications requiring long-lived HTTP/WebSocket connections.
- Workloads needing many persistent database connections without an appropriate serverless-compatible data layer.

Serverless is a useful execution model for specific workloads, not a universal replacement for traditional servers.

15. Mental models and final takeaways

The lecture closes with several principles:

- **Start with the problem.** Measure whether the system is slow, how slow it is, and which component is responsible.
- **Avoid premature optimization.** Do not introduce caching, sharding, microservices, or Kubernetes just because they are popular.
- **Prefer simple solutions.** A larger server, a suitable database index, or a monolith may be better than adding complex infrastructure.
- **Scale for the workload you actually have.** Do not build for a hypothetical million users on day one. Keep reasonable headroom and evolve based on observed traffic.
- **Build observability early.** Logs, metrics, traces, and load/performance tests help identify bottlenecks and provide warning before failures.
- **Treat complexity as a cost.** Every additional component must be understood, monitored, debugged, secured, and operated.
- **Performance is an ongoing engineering practice.** Build systems that can be measured, diagnosed, and recovered when real problems occur.

The lecture mentions tools such as Prometheus, Grafana, and New Relic as ways to observe systems.

16. Interview questions and answers

1. Why is statelessness important for horizontal scaling?

Any instance can handle any request because shared state is externalized. A request does not depend on private memory or files on one particular server.

2. How can in-memory sessions break a horizontally scaled application?

A user may authenticate on instance A, but a later request may reach B, which cannot see A's local session and may return 401. Store sessions in a shared store.

3. What does a load balancer do?

It receives incoming requests, selects a healthy backend instance, forwards the request, and returns the response.

4. Round robin vs least connections?

Round robin distributes requests in a fixed rotation; least connections prefers the instance with fewer active connections.

5. Why are health checks necessary?

They help the load balancer stop routing user traffic to unhealthy instances and restore instances after they recover.

6. What is a read replica?

A database copy that commonly serves read queries while the primary handles writes.

7. What is replication lag?

The delay between a change being committed on the primary and becoming visible on a replica.

8. How can read-after-write stale data be avoided?

Route relevant reads to the primary, wait for replication when appropriate, or use another consistency strategy suited to the workflow.

9. What is sharding?

Splitting a dataset across multiple physical database instances according to a sharding key.

10. Read replicas vs sharding?

Replicas copy the dataset to scale reads; sharding divides the dataset so different instances own different subsets.

11. What is a CDN PoP?

A geographically distributed edge location that can serve cached content close to users.

12. CDN vs edge computing?

A traditional CDN primarily serves cached content; edge computing also executes lightweight processing at the edge.

13. Why can't all backend logic run at the edge?

Edge runtimes have resource and runtime constraints and are designed for lightweight, low-latency operations.

14. How does asynchronous processing reduce perceived latency?

The API performs essential immediate work, enqueues non-critical work, and responds without waiting for the background task to finish.

15. What is the difference between a monolith and microservices?

A monolith is one deployable application; microservices split functionality into independently deployable services communicating over a network.

16. Are microservices always better for scaling?

No. They enable independent deployment/scaling but add network, consistency, debugging, and operational complexity.

17. What is autoscaling and why can it fail to prevent overload?

Autoscaling adjusts capacity based on signals, but it is reactive and new instances take time to start.

18. Does serverless mean there are no servers?

No. The provider manages the underlying execution infrastructure; developers mainly supply functions and event mappings.

19. What is a cold start?

The startup delay before a function executes in a newly initialized serverless environment.

20. When is serverless useful?

For event-driven or intermittent tasks such as image/video processing, queue jobs, and file-upload-triggered workflows.

17. Quick revision sheet

- **Horizontal scaling:** add more application instances.
- **Statelessness:** no request depends on private state held by one instance.
- Sessions → shared Redis/store; files → shared object storage; application data → shared database.
- **Load balancer:** distributes requests among backend instances.
- Algorithms: round robin, weighted round robin, least connections, least response time, resource-based.
- **Health checks:** detect unhealthy instances and remove them from normal traffic.
- **Read replica:** copy of primary, commonly used for reads.
- **Replication lag:** replica may temporarily return stale data.
- **Sharding:** split data across physical database instances using a sharding key.
- **CDN:** cache content at nearby PoPs to reduce latency and origin load.
- **Edge computing:** run lightweight logic close to the user.
- **Async processing:** enqueue non-immediate work for workers.
- **Monolith:** one deployable unit; can still scale horizontally.
- **Microservices:** independently deployable services; useful for clear team boundaries and independent scaling, but operationally complex.
- **Autoscaling:** automatically adjusts capacity, but reacts with a startup delay.
- **Serverless:** provider-managed function execution; cold starts and runtime limits matter.
- **Core principle:** measure first, choose the simplest adequate solution, and scale for observed needs.

18. Scope and source-faithfulness notes

These notes are based on the supplied Lecture 22 transcript, which explicitly continues the previous scaling/performance lecture. Numeric examples (latencies, utilization, percentages, developer counts, cold-start times, and costs) are teaching illustrations rather than universal targets. The transcript introduces concepts and trade-offs rather than providing complete production implementation guides for load balancers, distributed databases, edge runtimes, microservices, or serverless platforms.